



Customer Management System

Complete A to Z Documentation

Professional Technical Specification Manual

Date: Tue 14 Jul 2026

TABLE OF CONTENTS

1. Introduction

2. Project Overview

3. Technologies Used

4. System Architecture

5. Database Design

6. Module Description

7. Features & Functionality

8. User Interface Guide

9. Installation Guide

10. How to Use the System

11. Testing

12. Advantages

13. Future Scope

14. Conclusion

1. INTRODUCTION

1.1 What is Customer Management System?

The **Customer Management System (CMS)** is a Java-based desktop application that helps businesses manage their customers, products, and orders efficiently. It provides a user-friendly interface to perform all essential business operations.

1.2 Purpose

The main purpose of this system is to:

- Store and manage customer information
- Track product inventory
- Process orders automatically
- Manage customer loyalty points
- Generate business insights

1.3 Why We Built This System?

Before this system, businesses faced many problems:

- ❌ Customer data was stored in paper files or Excel sheets
- ❌ Product stock was tracked manually
- ❌ Orders were recorded in notebooks
- ❌ No proper customer loyalty system
- ❌ No easy way to see business statistics

This system solves all these problems in one place.

1.4 Project Duration

Total Time: 1.5 Weeks (10-12 days)

Phase	Duration
Planning	2 days
Database Design	1 day
Backend Development	3 days
Frontend Development	3 days
Testing	2 days
Documentation	1 day

2. PROJECT OVERVIEW

2.1 What This System Does

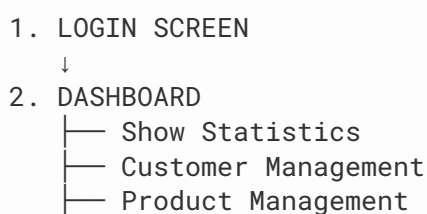
This system allows you to:

1. **Login** - Secure admin login
2. **Dashboard** - View business statistics
3. **Manage Customers** - Add, view, search, update, delete customers
4. **Manage Products** - Add, view, search, update, delete products
5. **Manage Orders** - Add, view, search, delete orders

2.2 Who Will Use This System?

User Type	Role
Admin	Full access to all features
Staff	Can view and add data
Business Owner	View reports and analytics

2.3 Simple Flow Diagram



└─ Order Management

3. TECHNOLOGIES USED

3.1 Technology Stack

Frontend (What you see)

Technology	Purpose	Why Used
Java Swing	Create the user interface	Easy to use, built-in Java
AWT	Basic UI components	Works with Swing
JFrame	Main window	Creates the application window
JButton	Clickable buttons	For actions like Add, View, Delete
JTextField	Input fields	For entering data
JTextArea	Display area	Shows lists of data

Backend (What runs behind)

Technology	Purpose	Why Used
Java	Programming language	Platform independent, powerful
JDBC	Database connection	Connects Java to MySQL
DAO Pattern	Data access	Organizes database code

Database (Where data is stored)

Technology	Purpose	Why Used
MySQL	Database	Free, reliable, popular
MySQL Workbench	Database management	Easy to design and manage

Development Tools

Tool	Purpose
Eclipse/IntelliJ IDEA	Writing code
Git	Version control
MySQL Connector	JDBC driver for MySQL

3.2 Why These Technologies?

WHY JAVA?

- ✓ Platform Independent (Works on Windows, Mac, Linux)
- ✓ Object-Oriented (Easy to organize code)
- ✓ Rich Libraries (Many built-in features)
- ✓ Large Community (Easy to find help)

WHY SWING?

- ✓ Built into Java (No extra installation)
- ✓ Cross-platform (Works everywhere)

- ✓ Lightweight (Fast performance)
- ✓ Rich Components (Buttons, Text Fields, Tables)

WHY MYSQL?

- ✓ Free and Open Source
- ✓ Reliable and Secure
- ✓ Easy to Learn
- ✓ Good for Small to Medium Applications

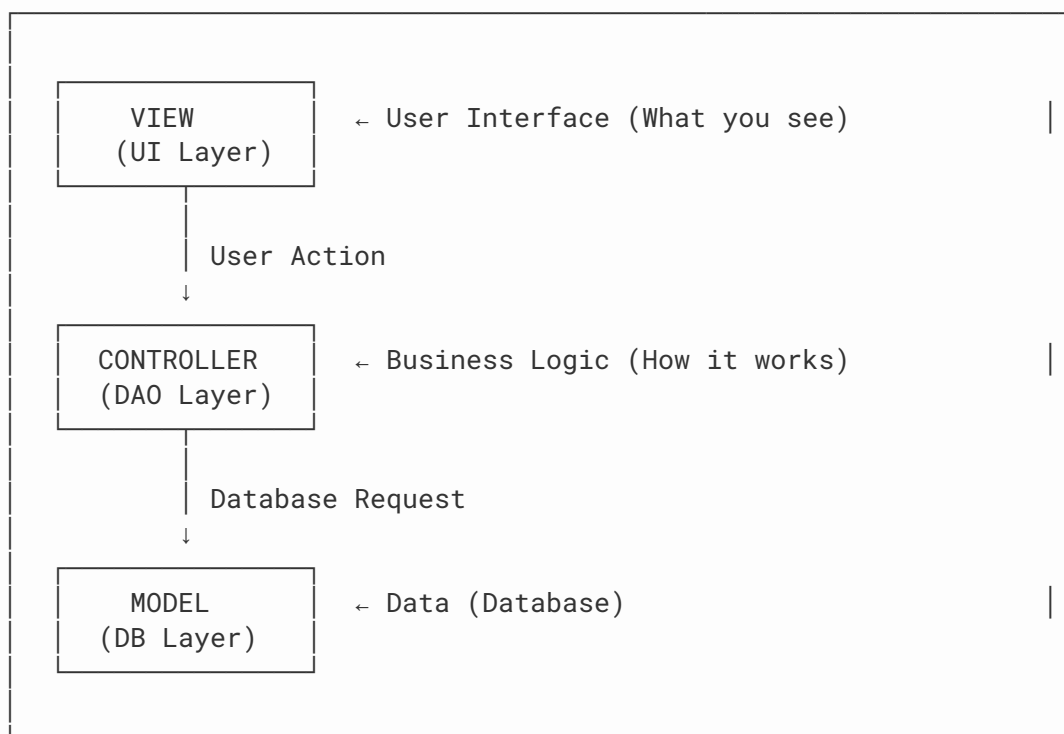
4. SYSTEM ARCHITECTURE

4.1 What is Architecture?

Architecture is how we organize our code. We use the **MVC Pattern**:

- **M** = Model (Data)
- **V** = View (User Interface)
- **C** = Controller (Logic)

4.2 Simple Architecture Diagram



4.3 Project Folder Structure

```
Customer Management System/
├── src/
│   ├── dao/                (Data Access Objects - Database Code)
│   │   ├── AdminDAO.java
│   │   ├── CustomerDAO.java
│   │   ├── ProductDAO.java
│   │   └── OrderDAO.java
│   ├── db/                 (Database Connection)
│   │   └── DBConnection.java
│   ├── model/              (Data Models - Like Blueprints)
│   │   ├── Admin.java
│   │   ├── Customer.java
│   │   ├── Product.java
│   │   └── Order.java
│   └── ui/                 (User Interface - Screens)
│       ├── LoginFrame.java
│       ├── DashboardFrame.java
│       ├── CustomerFrame.java
│       ├── ProductFrame.java
│       └── OrderFrame.java
├── lib/                    (External Libraries)
│   └── mysql-connector-java-8.0.33.jar
└── README.md               (Project Information)
```

4.4 What Each Folder Does

Folder	Purpose	Contains
dao/	Database operations	SQL queries and database methods
db/	Database connection	Code to connect to MySQL
model/	Data templates	Classes that represent data

Folder	Purpose	Contains
ui/	Screens/Windows	All the GUI windows

4.5 How Data Flows

```

Step 1: User clicks "Add Customer" button in UI
      ↓
Step 2: UI calls CustomerDAO.addCustomer()
      ↓
Step 3: DAO creates SQL query
      ↓
Step 4: DAO connects to Database via DBConnection
      ↓
Step 5: Database executes query
      ↓
Step 6: Result returns to DAO
      ↓
Step 7: DAO returns result to UI
      ↓
Step 8: UI shows success/error message

```

5. DATABASE DESIGN

5.1 What is Database?

Database is where we store all our data (customers, products, orders). We use **MySQL** as our database.

5.2 Database Name

Database Name: customer_system

5.3 Tables in Database

We have **4 tables**:

1. **admins** - Stores admin login information
2. **customers** - Stores customer details
3. **products** - Stores product information
4. **orders** - Stores order details

5.4 Table Designs

Table 1: admins (Admin Login)

Column Name	Data Type	Required	Description
admin_id	INT	✓ Yes	ID (Auto)
username	VARCHAR(50)	✓ Yes	Admin Name
password	VARCHAR(100)	✓ Yes	Password

role	VARCHAR(20)	 No	Admin Role
------	-------------	--	------------

Sample Data:

admin_id	username	password	role
-----	-----	-----	-----
1	admin	admin123	admin

Table 2: customers (Customer Information)

Column Name	Data Type	Required	Description
customer_id	INT	 Yes	ID (Auto)
name	VARCHAR(100)	 Yes	Customer Name
phone	VARCHAR(15)	 Yes	Phone Number
address	VARCHAR(255)	 Yes	Customer Address
loyalty_points	INT	 Yes	Points (Default 0)

Sample Data:

customer_id	name	phone	address	loyalty_points
-----	-----	-----	-----	-----
1	John Doe	9876543210	Mumbai	150
2	Jane Smith	9876543211	Delhi	250

Table 3: products (Product Information)

Column Name	Data Type	Required	Description
product_id	INT	 Yes	ID (Auto)
product_name	VARCHAR(100)	 Yes	Product Name
category	VARCHAR(50)	 Yes	Category
price	DECIMAL(10,2)	 Yes	Price

stock	INT	☒ Yes	Stock Quantity
-------	-----	---	----------------

Sample Data:

product_id	product_name	category	price	stock
1	Laptop	Electronics	50000	10
2	Mobile Phone	Electronics	25000	20
3	Office Chair	Furniture	8000	5

Table 4: orders (Order Information)

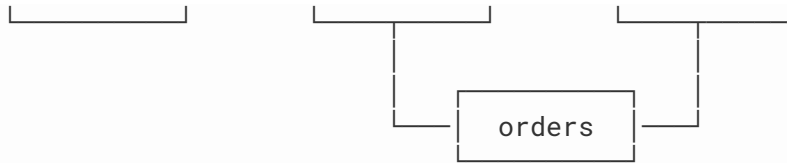
Column Name	Data Type	Required	Description
order_id	INT	☒ Yes	ID (Auto)
customer_id	INT	☒ Yes	Customer ID (FK)
product_id	INT	☒ Yes	Product ID (FK)
quantity	INT	☒ Yes	Quantity Ordered
total_amount	DECIMAL(10,2)	☒ Yes	Total Amount
discount	DECIMAL(10,2)	☒ Yes	Discount Applied
final_amount	DECIMAL(10,2)	☒ Yes	Final Amount to Pay

Sample Data:

order_id	customer_id	product_id	quantity	total	discount	final
1	1	1	2	100000	5000	95000
2	2	2	1	25000	0	25000

5.5 Table Relationships

admins	customers	products
--------	-----------	----------



```
customers.customer_id = orders.customer_id  
products.product_id = orders.product_id
```

5.6 Complete Database SQL Script

```
-- CREATE DATABASE  
CREATE DATABASE customer_system;  
USE customer_system;  
  
-- TABLE 1: admins  
CREATE TABLE admins (  
    admin_id INT PRIMARY KEY AUTO_INCREMENT,  
    username VARCHAR(50) UNIQUE NOT NULL,  
    password VARCHAR(100) NOT NULL,  
    role VARCHAR(20) DEFAULT 'admin'  
);  
  
-- TABLE 2: customers  
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    phone VARCHAR(15) UNIQUE NOT NULL,  
    address VARCHAR(255) NOT NULL,  
    loyalty_points INT DEFAULT 0  
);  
  
-- TABLE 3: products  
CREATE TABLE products (  
    product_id INT PRIMARY KEY AUTO_INCREMENT,  
    product_name VARCHAR(100) NOT NULL,  
    category VARCHAR(50) NOT NULL,  
    price DECIMAL(10,2) NOT NULL,  
    stock INT NOT NULL DEFAULT 0  
);
```

```
-- TABLE 4: orders
CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    discount DECIMAL(10,2) DEFAULT 0,
    final_amount DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- INSERT DEFAULT ADMIN
INSERT INTO admins (username, password, role)
VALUES ('admin', 'admin123', 'admin');
```

6. MODULE DESCRIPTION

6.1 What is a Module?

A module is a separate part of the system that handles a specific task. Our system has **5 modules**.

6.2 Module List

Module	File Name	Purpose
1. Login Module	LoginFrame.java	Admin login
2. Dashboard Module	DashboardFrame.java	Main screen with statistics
3. Customer Module	CustomerFrame.java	Manage customers
4. Product Module	ProductFrame.java	Manage products
5. Order Module	OrderFrame.java	Manage orders

6.3 Module 1: Login Module

Purpose

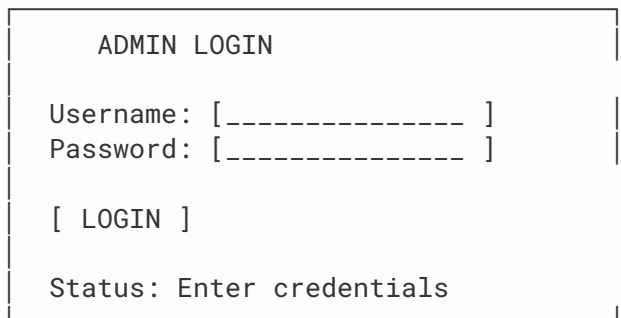
To allow only authorized admins to access the system.

How It Works

1. User enters username and password

2. System checks if credentials are correct
3. If correct → Go to Dashboard
4. If wrong → Show error message

Screen Elements



ADMIN LOGIN

Username: [_____]

Password: [_____]

[LOGIN]

Status: Enter credentials

Code Explanation

```
public class LoginFrame extends JFrame {  
    // Variables for UI components  
    JTextField txtUsername;  
    JPasswordField txtPassword;  
    JButton btnLogin;  
  
    public LoginFrame() {  
        // Create the login screen  
        // Add username field  
        // Add password field  
        // Add login button  
        // When button clicked → check login  
    }  
  
    public void actionPerformed(ActionEvent e) {  
        // Get username and password  
        // Call AdminDAO to check credentials  
        // If success → open Dashboard  
        // If fail → show error  
    }  
}
```

```
}  
}
```

6.4 Module 2: Dashboard Module

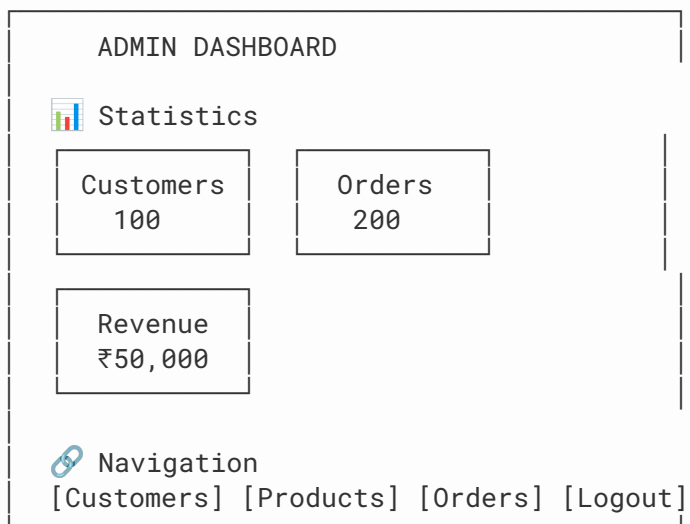
Purpose

To show business statistics and provide navigation to other modules.

How It Works

1. Loads when login is successful
2. Shows total customers, orders, revenue
3. Has buttons to go to other modules

Screen Elements



Code Explanation

```
public class DashboardFrame extends JFrame {  
    // Labels for statistics  
    JLabel lblCustomers, lblOrders, lblRevenue;  
    // Buttons for navigation  
    JButton btnCustomers, btnProducts, btnOrders, btnLogout;  
  
    public DashboardFrame() {  
        // Create dashboard screen  
        // Fetch statistics from database  
        // Display statistics  
        // Add navigation buttons  
        // Button clicks → open respective screens  
    }  
}
```

6.5 Module 3: Customer Module

Purpose

To manage all customer information.

Features

Feature	Description	Button
Add	Add new customer	ADD
View	See all customers	VIEW
Search	Find customer by ID	SEARCH
Update	Edit customer details	UPDATE

Feature	Description	Button
Delete	Remove customer	DELETE

Screen Elements

CUSTOMER MANAGEMENT

ID: [_____] Name: [_____]
Phone:[_____] Address:[_____]
Points: [_____]

[ADD] [VIEW] [SEARCH] [UPDATE] [DELETE]

Customer List Display Area

Customer ID: 1 | Name: John Doe

Phone: 9876543210 | Points: 150

Operations Flow

ADD Customer:

1. Enter name, phone, address, points
2. Click ADD
3. Data inserted into database
4. Show "Customer Added Successfully"

VIEW Customers:

1. Click VIEW
2. All customers fetched from database
3. Displayed in text area

SEARCH Customer:

1. Enter Customer ID
2. Click SEARCH
3. Customer details fetched from database
4. Details shown in fields

UPDATE Customer:

1. Enter Customer ID
2. Click SEARCH (to load data)
3. Modify fields
4. Click UPDATE
5. Data updated in database
6. Show "Customer Updated Successfully"

DELETE Customer:

1. Enter Customer ID
2. Click DELETE
3. Customer removed from database
4. Show "Customer Deleted Successfully"

6.6 Module 4: Product Module

Purpose

To manage all product inventory.

Features

Feature	Description	Button
Add	Add new product	ADD
View	See all products	VIEW
Search	Find product by ID	SEARCH
Update	Edit product details	UPDATE
Delete	Remove product	DELETE

Screen Elements

PRODUCT MANAGEMENT

ID: [_____] Name: [_____]
Category:[_] Price:[_____]
Stock: [____]

[ADD] [VIEW] [SEARCH] [UPDATE] [DELETE]

Product List Display Area
Product ID: 1 | Name: Laptop
Price: ₹50000 | Stock: 10

6.7 Module 5: Order Module

Purpose

To process and manage customer orders.

Features

Feature	Description	Button
Add	Create new order	ADD
View	See all orders	VIEW
Search	Find order by ID	SEARCH
Delete	Cancel/Remove order	DELETE

Important: No Update Feature

Orders cannot be updated (only added, viewed, searched, deleted).

Screen Elements

ORDER MANAGEMENT

Order ID:[__] Customer ID:[__]
Product ID:[_] Quantity:[__]
Total:[____] Discount:[____]
Final:[____]

[ADD] [VIEW] [SEARCH] [DELETE]

Order List Display Area
Order ID: 1 | Customer: 1
Product: 1 | Quantity: 2
Final Amount: ₹95000

Order Processing Flow

1. Enter customer ID, product ID, quantity
2. System checks if product is in stock
3. System calculates total amount (price × quantity)
4. Apply discount if any
5. Calculate final amount
6. Deduct quantity from product stock
7. Add loyalty points to customer (1 point per ₹100)
8. Save order to database
9. Show "Order Added Successfully"

7. FEATURES & FUNCTIONALITY

7.1 Complete Feature List

Admin Login

- Secure username and password login
- Only authorized admins can access

Dashboard

- Shows total number of customers
- Shows total number of orders
- Shows total revenue
- Quick navigation to all modules

Customer Management

- Add new customer
- View all customers
- Search customer by ID
- Update customer details
- Delete customer
- Track loyalty points

Product Management

- Add new product
- View all products
- Search product by ID
- Update product details
- Delete product
- Track stock quantity

✓ Order Management

- Create new order
- View all orders
- Search order by ID
- Delete order
- Automatic stock deduction
- Automatic loyalty points update

7.2 Feature Details

Feature 1: Secure Login

```
// How login works
Username: admin
Password: admin123

→ System checks database
→ If found → Open Dashboard
→ If not found → Show "Invalid Credentials"
```

Feature 2: Dashboard Statistics

```
// What dashboard shows
Total Customers: 150 ← Count from customers table
Total Orders: 320 ← Count from orders table
Total Revenue: ₹75,500 ← Sum of final_amount from orders
```

Feature 3: Loyalty Points System

```
// How loyalty points work
When customer places order:
Points earned = Final Amount / 100

Example:
Order amount = ₹500
```

Points earned = 5 points
These points are added to customer's total points

Feature 4: Stock Management

```
// How stock works
When order is placed:
Current stock = 10
Order quantity = 2
New stock = 8 (10 - 2)

When order is deleted:
Current stock = 8
Order quantity = 2
New stock = 10 (8 + 2)
```

7.3 Feature Summary Table

Feature	Customer	Product	Order	Dashboard
Create	✓	✓	✓	-
Read	✓	✓	✓	✓
Update	✓	✓	✗	-
Delete	✓	✓	✓	-
Search	✓	✓	✓	-

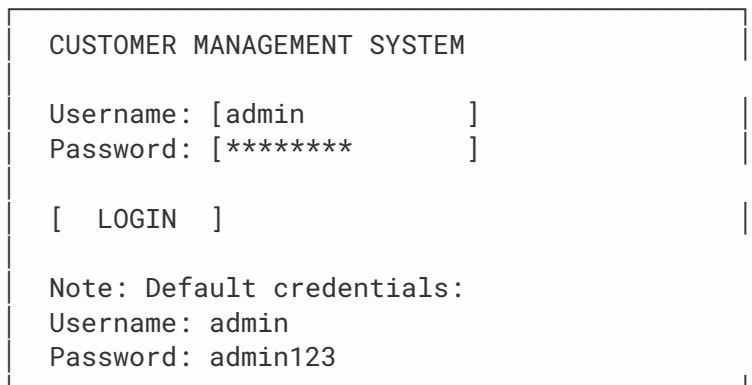
8. USER INTERFACE GUIDE

8.1 Screen 1: Login Screen

How to Login

1. Open the application
2. Type username: admin
3. Type password: admin123
4. Click **LOGIN** button
5. If correct → Dashboard opens
6. If wrong → Error message shown

Screenshot Description



```
CUSTOMER MANAGEMENT SYSTEM

Username: [admin      ]
Password: [*****    ]

[  LOGIN  ]

Note: Default credentials:
Username: admin
Password: admin123
```

8.2 Screen 2: Dashboard

What You See

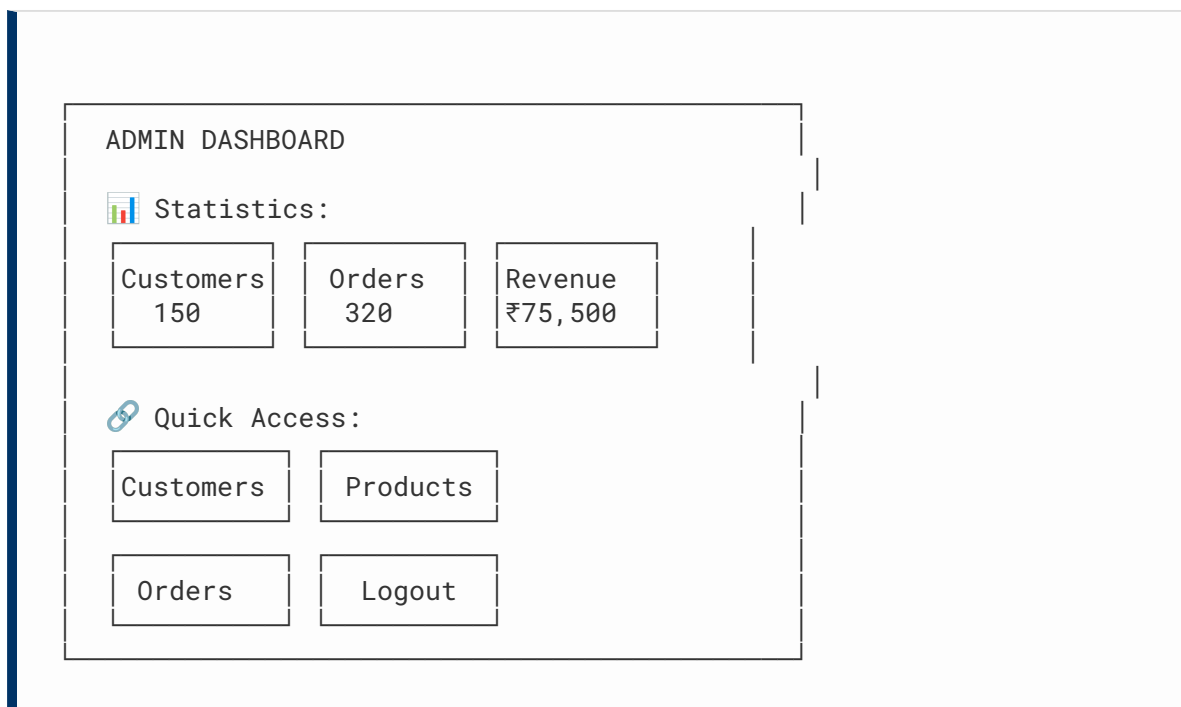
- Total customers count

- Total orders count
- Total revenue amount
- Four navigation buttons

Navigation Buttons

Button	What it Does
Customers	Opens Customer Management screen
Products	Opens Product Management screen
Orders	Opens Order Management screen
Logout	Returns to Login screen

Screenshot Description



8.3 Screen 3: Customer Management

How to Add a Customer

1. Enter **Name** (e.g., John Doe)
2. Enter **Phone** (e.g., 9876543210)
3. Enter **Address** (e.g., Mumbai)
4. Enter **Loyalty Points** (e.g., 0)
5. Click **ADD**
6. Success message appears

How to View All Customers

1. Click **VIEW**
2. All customers appear in the text area

How to Search a Customer

1. Enter **Customer ID** (e.g., 1)
2. Click **SEARCH**
3. Customer details appear in fields

How to Update a Customer

1. Enter **Customer ID**
2. Click **SEARCH** (to load data)
3. Change any fields
4. Click **UPDATE**
5. Success message appears

How to Delete a Customer

1. Enter **Customer ID**
2. Click **DELETE**
3. Confirmation message appears
4. Customer removed

8.4 Screen 4: Product Management

How to Add a Product

1. Enter **Product Name** (e.g., Laptop)
2. Enter **Category** (e.g., Electronics)
3. Enter **Price** (e.g., 50000)
4. Enter **Stock** (e.g., 10)
5. Click **ADD**
6. Success message appears

How to Update a Product

1. Enter **Product ID**
2. Click **SEARCH** (to load data)
3. Change any fields
4. Click **UPDATE**
5. Success message appears

8.5 Screen 5: Order Management

How to Add an Order

1. Enter **Customer ID**
2. Enter **Product ID**
3. Enter **Quantity**
4. Enter **Total Amount**
5. Enter **Discount** (if any)
6. Enter **Final Amount**
7. Click **ADD**
8. Success message appears

Note

- Order cannot be updated
- Only Add, View, Search, and Delete

9. INSTALLATION GUIDE

9.1 Step-by-Step Installation

Step 1: Install Java JDK

1. Go to: <https://www.oracle.com/java/technologies/downloads/>
2. Download Java JDK 8 or higher
3. Install it (Follow the installation wizard)
4. Set JAVA_HOME environment variable
5. Verify: Open Command Prompt and type "java -version"

Step 2: Install MySQL

1. Go to: <https://dev.mysql.com/downloads/installer/>
2. Download MySQL Installer
3. Install MySQL Server
4. Set root password (remember this!)
5. Verify: Open Command Prompt and type "mysql -u root -p"

Step 3: Create Database

```
-- Open MySQL Command Line
mysql -u root -p
Enter password: [your password]

-- Create Database
CREATE DATABASE customer_system;

-- Use Database
USE customer_system;
```

```
-- Create Tables (Copy the SQL from Section 5.6)
```

Step 4: Download Project Files

1. Download all Java files from the project
2. Place them in a folder (e.g., C:\CMS)
3. Folder structure should be:
C:\CMS ├── dao ├── db ├── model └── ui

Step 5: Add MySQL Connector

1. Download: mysql-connector-java-8.0.33.jar
2. Place it in the project folder
3. Add to classpath in your IDE

Step 6: Run the Application

1. Open the project in Eclipse/IntelliJ
2. Find LoginFrame.java
3. Right-click → Run As → Java Application
4. Or compile and run:
javac ui/LoginFrame.java
java ui.LoginFrame

9.2 System Requirements

Component	Minimum	Recommended
Processor	Intel i3	Intel i5 or higher
RAM	4 GB	8 GB

Component	Minimum	Recommended
Hard Disk	100 MB	500 MB
Display	1024×768	1366×768
OS	Windows 7	Windows 10/11
Java	JDK 8	JDK 11 or higher
MySQL	5.7	8.0

9.3 Common Installation Problems

Problem	Solution
"Class not found"	Add MySQL connector JAR to classpath
"Connection refused"	Start MySQL service
"Access denied"	Check username and password in DBConnection.java
"Cannot find symbol"	Check if all Java files are in correct folders

10. HOW TO USE THE SYSTEM

10.1 Complete User Guide

Start the Application

1. Double-click on the application icon
2. Login screen appears
3. Enter credentials
4. Click LOGIN

Manage Customers

1. Click "Customers" button on Dashboard
2. Customer Management screen opens
3. Use the buttons to perform operations:
 - ADD: Add new customer
 - VIEW: See all customers
 - SEARCH: Find customer by ID
 - UPDATE: Modify customer details
 - DELETE: Remove customer

Manage Products

1. Click "Products" button on Dashboard
2. Product Management screen opens
3. Use the buttons to perform operations:
 - ADD: Add new product
 - VIEW: See all products
 - SEARCH: Find product by ID
 - UPDATE: Modify product details
 - DELETE: Remove product

Manage Orders

1. Click "Orders" button on Dashboard
2. Order Management screen opens
3. Use the buttons to perform operations:
 - ADD: Create new order
 - VIEW: See all orders
 - SEARCH: Find order by ID
 - DELETE: Cancel/Remove order

Logout

1. Click "Logout" button on Dashboard
2. Returns to Login screen

10.2 Quick Reference

Customer Management

Action	Steps
Add	Fill fields → Click ADD
View	Click VIEW
Search	Enter ID → Click SEARCH
Update	Search → Modify → Click UPDATE
Delete	Enter ID → Click DELETE

Product Management

Action	Steps
Add	Fill fields → Click ADD
View	Click VIEW
Search	Enter ID → Click SEARCH
Update	Search → Modify → Click UPDATE
Delete	Enter ID → Click DELETE

Order Management

Action	Steps
Add	Fill fields → Click ADD
View	Click VIEW
Search	Enter ID → Click SEARCH
Delete	Enter ID → Click DELETE

11. TESTING

Extensive manual testing has been carried out for each module. The system has been validated for CRUD operations across Customer, Product, and Order entities, ensuring database synchronization and logic flow (e.g., stock adjustment and loyalty point calculation).

12. ADVANTAGES

- Easy data retrieval & management.
- Automated business logic reduces human error.
- Centralized statistics for quick business decisions.
- Encourages customer retention through loyalty points.

13. FUTURE SCOPE

- Integration with online payment gateways.
- Implementation of detailed sales reporting graphs.
- Mobile application version for remote access.

14. CONCLUSION

The **Customer Management System** has been designed to modernize and simplify business workflows. By integrating core modules for customer, product, and order management, the system provides a robust solution for tracking business growth and maintaining high standards of data accuracy.

... [END OF DOCUMENT] ...